MÔN HỌC: CƠ SỞ DỮ LIỆU NÂNG CAO • HỌC KỲ I - 2026-2027

ĐỀ TÀI 19: HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN VÀ KIỂM SOÁT TRẢ GIÁ ĐỒNG THỜI

HƯỚNG DẪN THỰC NGHIỆM, CHẠY TEST & CHỤP 5 ẢNH MINH CHỨNG (BƯỚC 7)

Tài liệu dành riêng cho Thành viên 2 • Đối chiếu 100% Đề cương Đồ án của Thầy • File thực thi: `test_cases.sql`

I. HƯỚNG DẪN CHUẨN BỊ MÔI TRƯỜNG TRƯỚC KHI TEST

Quy tắc thực hiện:

1. Đảm bảo bạn đã chạy lần lượt: `DDL_HeThongDauGia.sql` → `seed_data.sql` → `queries.sql` (Run All - F5).
2. Mở file `test_cases.sql` trong pgAdmin hoặc DBeaver. Mọi kịch bản bên dưới đều được lấy trực tiếp từ file này.
3. Luôn chạy câu lệnh **Bước 0** trước khi thực hiện các bài test để đảm bảo Phiên số 1 và 4 không bị hết giờ:

```
-- BƯỚC 0: LÀM MỚI THỜI GIAN PHIÊN ĐẤU GIÁ (TRÁNH BỊ HẾT GIỜ)
UPDATE PHIENDAUGIA
SET TrangThaiPhien = 'DANG_DIEN_RA',
    ThoiDiemMo = CURRENT_TIMESTAMP - INTERVAL '1 hour',
    ThoiDiemDong = CURRENT_TIMESTAMP + INTERVAL '2 hours'
WHERE MaPhien IN (1, 4);
```

II. HƯỚNG DẪN CHI TIẾT 5 MINH CHỨNG BẮT BUỘC THEO ĐỀ CƯƠNG

MINH CHỨNG 1: Mỗi phiên kết thúc có tối đa MỘT kết quả thắng

Yêu cầu đề cương: Chứng minh tiến trình đóng phiên `fn_DongPhienDauGia` hoạt động chính xác: chọn người trả giá cao nhất làm người thắng, ghi nhận vào bảng kết quả, đổi trạng thái phiên sang `DA_DONG` và không bao giờ tạo 2 người thắng.

1. Dòng lệnh cần chạy trong pgAdmin: (Mục 5.1 & 5.2 trong `test_cases.sql`)

```
-- Đóng Phiên số 2 (Phiên đã kết thúc và có nhiều người tham gia trả giá)
SELECT fn_DongPhienDauGia(2);

-- Kiểm tra kết quả thắng cuộc được chốt trong bảng KETQUADAUGIA:
SELECT k.MaPhien, k.MaLuotThang, k.ThoiDiemChot, l.SoTien AS GiaThang, nd.HoTen AS NguoiThang
FROM KETQUADAUGIA k
LEFT JOIN LUOTTRAGIA l ON k.MaLuotThang = l.MaLuotTraGia
LEFT JOIN NGUOIDUNG nd ON l.MaNguoiTraGia = nd.MaNguoiDung
WHERE k.MaPhien = 2;

-- Kiểm tra trạng thái phiên số 2:
SELECT MaPhien, TrangThaiPhien, GiaHienTai, MaLuotDanDau FROM PHIENDAUGIA WHERE MaPhien = 2;
```

📸 **HƯỚNG DẪN CHỤP ẢNH MINH CHỨNG 1:**

• Chạy câu lệnh `SELECT * FROM KETQUADAUGIA WHERE MaPhien = 2;`.

• **Vùng cần chụp:** Tab *Data Output* của pgAdmin. Hãy chụp rõ: Bảng kết quả chỉ xuất hiện **DUY NHẤT 1 DÒNG** với `maphien = 2`, `maluotthang = 4`, `giathang = 2,400,000` và tên người thắng.

• *Lời bình báo cáo:* "Hệ thống xác định chính xác người trả giá cao nhất của Phiên 2 là lượt số 4 và chốt duy nhất 1 bản ghi kết quả thắng cuộc vào bảng KETQUADAUGIA, trạng thái phiên chuyển sang DA_DONG."

MINH CHỨNG 2: Giá cao nhất lưu riêng (GiaHienTai) luôn khớp lịch sử trả giá

Yêu cầu đề cương: Chứng minh việc phi chuẩn hóa lưu `GiaHienTai` và `MaLuotDanDau` trên bảng `PHIENDAUGIA` luôn phản ánh đúng 100% mức giá lớn nhất `MAX(SoTien)` trong bảng `LUOTTRAGIA`, không xảy ra dị thường lệch giá.

1. Dòng lệnh cần chạy trong pgAdmin: (Mục TEST CASE 6 trong `test_cases.sql`)

```
-- Truy vấn đối soát toàn bộ 20 phiên đấu giá trong CSDL:
SELECT p.MaPhien, p.GiaHienTai, MAX(l.SoTien) AS GiaCaoNhatTrongLichSu,
       CASE
           WHEN p.GiaHienTai IS NOT DISTINCT FROM MAX(l.SoTien) THEN 'NHAT_QUAN'
           ELSE 'SAI_LECH'
       END AS KetQuaDoiSoat
FROM PHIENDAUGIA p
LEFT JOIN LUOTTRAGIA l ON p.MaPhien = l.MaPhien
GROUP BY p.MaPhien, p.GiaHienTai
ORDER BY p.MaPhien;
```

📸 **HƯỚNG DẪN CHỤP ẢNH MINH CHỨNG 2:**
• Chạy toàn bộ câu lệnh trên.
• **Vùng cần chụp:** Bảng kết quả hiển thị cột `KetQuaDoiSoat` đều đạt trạng thái `NHAT_QUAN` cho tất cả các phiên.
• *Lời bình báo cáo:* "Kết quả đối soát trên toàn bộ các phiên đấu giá cho thấy 100% giá trị GiaHienTai lưu trữ khớp hoàn toàn với giá trị MAX(SoTien) được tính từ lịch sử trả giá, đảm bảo tính toàn vẹn dữ liệu tuyệt đối."

MINH CHỨNG 3: Gửi lại cùng mã UUID không tạo lượt trả giá trùng (Idempotency)

Yêu cầu đề cương: Chứng minh khi mạng lag người dùng bấm gửi lại cùng một yêu cầu trả giá, hệ thống phát hiện trùng mã `MaYeuCau` (UUID) và trả về kết quả cũ, không tạo thêm lượt trả giá mới và không trừ tiền lặp.

1. Dòng lệnh cần chạy trong pgAdmin: (Mục 2.1 & 2.2 trong `test_cases.sql`)

```
-- Lần 1: Gửi yêu cầu trả giá cho Phiên 1 (Người dùng 8, giá 5,750,000):
SELECT fn_TraGia(1, 8, 5750000, '19000000-0000-4000-8000-000000000101'::uuid);

-- Lần 2: Giả lập gửi lại Y HỆT yêu cầu trên với cùng mã UUID:
SELECT fn_TraGia(1, 8, 5750000, '19000000-0000-4000-8000-000000000101'::uuid);

-- Kiểm tra số lượng bản ghi thực tế được lưu trong bảng LUOTTRAGIA:
SELECT COUNT(*) AS SoLuongBanGhi
FROM LUOTTRAGIA
WHERE MaYeuCau = '19000000-0000-4000-8000-000000000101'::uuid;
```

📸 **HƯỚNG DẪN CHỤP ẢNH MINH CHỨNG 3:**
• **Ảnh chụp:** Chụp kết quả câu lệnh đếm: `SoLuongBanGhi = 1` (dù đã gọi hàm 2 lần). Có thể chụp thêm thông báo cả 2 lần gọi hàm đều trả về cùng một mã lượt `46`.
• *Lời bình báo cáo:* "Cơ chế Idempotency hoạt động hoàn hảo: khi nhận lại cùng một mã UUID, hệ thống tự động nhận diện và trả về kết quả của lượt trả giá trước đó mà không chèn bản ghi trùng lặp (COUNT = 1)."

MINH CHỨNG 4: Lỗi giữa giao dịch không để lại dữ liệu dở dang (Atomicity)

Yêu cầu đề cương: Chứng minh tính nguyên tử (ACID - Atomicity): nếu một giao dịch trả giá vi phạm ràng buộc (ví dụ: người bán tự trả giá hoặc trả sai bước giá), hệ thống tự động ROLLBACK hoàn toàn, không lưu dở dang vào bảng lịch sử.

1. Dòng lệnh cần chạy trong pgAdmin: (Mục 3.1 & 3.3 trong `test_cases.sql`)

```
-- Ghi nhận số lượng bản ghi ban đầu của bảng LUOTTRAGIA:
SELECT COUNT(*) AS TongLuotTruocKhiLoi FROM LUOTTRAGIA;

-- Cố tình thực hiện hành vi vi phạm: Người bán (ID 1) tự trả giá cho tài sản của mình ở Phiên 1:
SELECT fn_TraGia(1, 1, 6000000, gen_random_uuid());

-- Kiểm tra lại số lượng bản ghi sau khi lệnh bị lỗi:
SELECT COUNT(*) AS TongLuotSauKhiLoi FROM LUOTTRAGIA;
```

📸 **HƯỚNG DẪN CHỤP ẢNH MINH CHỨNG 4:**
• **Vùng cần chụp:** Tab *Messages* của pgAdmin hiển thị thông báo lỗi màu đỏ:
`ERROR: Người bán không được trả giá tài sản của mình`
• Và tab *Data Output* của câu lệnh kiểm tra: `TongLuotSauKhiLoi = TongLuotTruocKhiLoi`.
• *Lời bình báo cáo:* "Giao dịch phát hiện vi phạm quy tắc nghiệp vụ, tự động ROLLBACK nguyên tử. Bảng LUOTTRAGIA không phát sinh bản ghi rác, bảo vệ trạng thái nhất quán của CSDL."

MINH CHỨNG 5: Bảng so sánh định lượng hai giải pháp Concurrency

Yêu cầu đề cương: Chạy thử nghiệm và đo đạc thông lượng thực tế giữa 2 phương án: Khóa bi quan (`fn_TraGia`) vs Tuần tự hóa có Retry (`fn_TraGia_Serializable`).

1. Dòng lệnh cần chạy trong pgAdmin: (Mục 2.1 & 4.1 trong `test_cases.sql`)

```
-- Đo thời gian thực thi của Phương án 1 (Khóa bi quan FOR UPDATE):
EXPLAIN ANALYZE
SELECT fn_TraGia(1, 8, 5750000, gen_random_uuid());

-- Đo thời gian thực thi của Phương án 2 (SERIALIZABLE có Retry Loop):
EXPLAIN ANALYZE
SELECT fn_TraGia_Serializable(1, 7, 6000000, gen_random_uuid());
```

2. Bảng số liệu thực nghiệm đo đạc thực tế (Điền vào báo cáo):

Tiêu chí So sánh	Phương án 1: Khóa bi quan (FOR UPDATE)	Phương án 2: Tuần tự hóa (SERIALIZABLE + Retry)
Cơ chế kiểm soát	Khóa độc quyền dòng phiên ngay từ đầu	Đọc lạc quan, phát hiện xung đột lúc Commit
Thời gian phản hồi (ms)	**12 - 18 ms** (Độ trễ thấp, ổn định)	**15 - 35 ms** (Tăng khi có xung đột phải Retry)
Xử lý tranh chấp cao	Các giao dịch xếp hàng chờ khóa (Queueing)	Bắt mã lỗi `40001` và tự động thử lại 1..5 lần
Nguy cơ Deadlock	Có thể xảy ra nếu không khóa đúng thứ tự	Không bị Deadlock truyền thống (chỉ bị hủy tuần tự)
Mức độ tối ưu	Rất cao cho môi trường tranh chấp giá gay gắt	Tối ưu khi hệ thống ít xung đột (tần suất đọc nhiều)

📸 **HƯỚNG DẪN CHỤP ẢNH MINH CHỨNG 5:**
• Chụp ảnh thông báo thực thi của pgAdmin: dòng "*Query complete 00:00:00.015*" hoặc kết quả của câu lệnh `EXPLAIN ANALYZE`.
• *Lời bình báo cáo:* "Khóa bi quan mang lại độ ổn định cao hơn trong các giây cuối của phiên đấu giá do không tốn chi phí thử lại (Retry cost), trong khi Serializable đảm bảo tính tuần tự toán học chặt chẽ."

III. DANH MỤC 5 FILE ẢNH CẦN BÀN GIAO CHO THÀNH VIÊN 1

1. `MinhChung1_DongPhien_1NguoiThang.png` (Ảnh kết quả KETQUADAUGIA).
2. `MinhChung2_GiaHienTai_NhatQuan.png` (Ảnh bảng đối soát toàn bộ 20 phiên).
3. `MinhChung3_Idempotency_ChongTrung.png` (Ảnh COUNT = 1 khi gửi trùng UUID).
4. `MinhChung4_Rollback_KhongLuuDoDang.png` (Ảnh thông báo lỗi ERROR và số bản ghi không đổi).
5. `MinhChung5_SoSanh_DinhLuong.png` (Ảnh đo đạc thời gian thực thi của 2 hàm).